INTERNSHIP TECHNICAL CHALLENGE

Solar Panel Image Capture and Condition Mapping

Intended for	Internship candidates in Robotics, Computer Vision, and Software
Format	Remote and asynchronous, followed by a technical defense session
Estimated effort	10 to 12 hours of focused work
Deadline	**July 5 at 10:00 PM PT**
Questions	spuentes.dev@sunnybotics.com, during the first 48 hours only

1. The Challenge

At Sunnybotics, our robots travel through solar farms capturing images of the panels to support cleaning, inspection, and maintenance decisions. Your challenge is to build a working prototype that receives or simulates those captures, links each image to its location and context, analyzes the condition of the panel, and produces traceable results.

In the end, every image must answer three questions: where was it taken, what diagnosis did the system produce, and what visual evidence supports that diagnosis? We are not looking for a perfect model or a production platform. We want to see how you think, how you structure a system from end to end, and how you justify your decisions when working with imperfect data.

2. What We Evaluate

- **Execution:** a working, reproducible pipeline from ingestion to results.
- **Technical judgment:** choosing and justifying the right technique (OpenCV, heuristics, classical machine learning, pretrained models, or lightweight deep learning).
- **Data modeling:** a consistent, traceable structure for images, metadata, and results.
- **Handling uncertainty:** taking GPS error, shadows, glare, and unusable images seriously.
- **Communication:** explaining decisions, assumptions, and limitations clearly.

3. Required Features

Code	Feature	Acceptance criteria
RF-01	**Image ingestion**	The system receives or processes images (local folder, web interface, API, or route simulation). Every image has a unique identifier.

RF-02	**Metadata**	Every image is linked to image_id, timestamp, latitude, longitude, robot_id, mission_id, panel_row, and panel_id. If you simulate coordinates, document how you generated them.
RF-03	**Condition analysis**	The system classifies each image into at least 3 of these categories: clean, dirty, shadowed, damaged, glare, plus the required uncertain category. Each result includes a confidence level.
RF-04	**Priority score**	A cleaning or inspection priority score from 0 to 100 (0 to 20 low, 21 to 50 medium, 51 to 80 high, 81 to 100 critical), with a justified formula.
RF-05	**Annotated image**	An annotated version of every processed image (boxes, masks, or text with condition, confidence, and score), saved in outputs/annotated/.
RF-06	**Spatial association**	It must be possible to query where each image was taken and which row and panel it belongs to. Explain how you guarantee the correct association despite GPS error.
RF-07	**Export and visualization**	Results in results.json, .csv, or .geojson with the fields listed in deliverable E2, plus at least one visualization: a map, table, dashboard, or farm grid.

The system must not crash on a missing or corrupted image, incomplete metadata, or invalid coordinates. Reasonable, documented handling is enough.

4. Technical Rules and Data

- **Free choice of stack:** use whatever you prefer. Suggestions: Python, OpenCV, Pandas, Scikit-learn, Streamlit, FastAPI, and Leaflet or Mapbox for maps.
- **Simple, reproducible execution:** the project runs with 1 or 2 commands documented in the README, with dependencies declared (requirements.txt or equivalent).
- **Data:** use the dataset provided by Sunnybotics [link provided at delivery] or public images with a documented source and license. Metadata may be simulated as long as you state your assumptions.
- **Process rules:** you may use AI assistants if you disclose it in the README, but you must be able to defend every decision during the technical defense. The work must be individual. Working code is mandatory; a slide deck is not a submission. Documenting where your system fails earns points, and hiding it costs points.

5. Deliverables

ID	Deliverable	Minimum content

E1	**Repository**	Organized source code, dependencies, and a README covering installation, execution, the pipeline, sample outputs, and known limitations.
E2	**Structured results**	Per image: image_id, timestamp, latitude, longitude, robot_id, mission_id, panel_row, panel_id, condition, confidence, cleaning_priority_score, annotated_image_path, detected_issues.
E3	**Annotated images**	An outputs/annotated/ folder with every processed image.
E4	**Visualization**	A map, table, dashboard, or grid showing what was captured, where, and with what result.
E5	**Technical report**	Two pages maximum: approach, how images are matched to panels, the score formula, assumptions, failure cases, improvements, and the answers to section 6.
E6	**Final video**	3 minutes maximum. See section 8.

6. Required Questions (answer in the report)

1. How do you guarantee that each photo is linked to the correct panel, even with a GPS error of 1 to 3 meters?
2. How do you detect that an image is unusable for analysis (blurry, overexposed, panel not visible)?
3. How would you tell real dirt apart from a temporary shadow?
4. What metrics would you use to evaluate your system without complete ground truth?
5. Which parts of your solution would run on the robot and which in the cloud? Why?
6. What is the biggest technical risk in your solution, and how would you scale it to thousands of panels?

7. Evaluation (100 points)

Dimension	Weight	What the evaluator looks for
D1. Ingestion and metadata (RF-01, RF-02)	**15**	A reproducible flow, unique identifiers, and complete, consistent metadata.
D2. Image, location, and panel association (RF-06)	**20**	A clear data model and a serious discussion of GPS error and spatial uncertainty.
D3. Visual analysis (RF-03)	**20**	A technique applied with judgment, correct use of uncertain, and sound reasoning behind the approach.
D4. Priority score (RF-04)	**10**	A reasonable, justified formula connected to real operational decisions.

D5. Annotated images (RF-05)	**10**	Clear annotations, consistent with the diagnosis and useful for human review.
D6. Export and visualization (RF-07)	**10**	Correct outputs and full traceability across image, location, and diagnosis.
D7. Code quality	**10**	Structure, naming, separation of concerns, and ease of execution.
D8. Report and video	**5**	Clarity, honesty about limitations, and solid answers to the required questions.
Total	**100**	Bonus of up to 10 extra points (capped at 100): a simulated robot route, a dashboard with filters, an interactive map, automatic detection of unusable images, or decision logic for the robot.

- **Performance levels per dimension:** 0 to 39% of the weight: missing or not working. 40 to 69%: works but lacks justification. 70 to 89%: works, is reproducible, and is justified. 90 to 100%: also shows extra judgment (edge cases, real field conditions).
- **Thresholds:** a score of 70 or higher advances to the technical defense, a remote session of 30 minutes. Scores between 60 and 69 advance at the team's discretion. Scores below 60 do not continue.

8. Final Video (3 minutes maximum)

Close your submission with a video of 3 minutes maximum in which you appear on camera while explaining your work: what you built and why you built it that way, the decisions you made, the alternatives you discarded, and what you would improve. You must show the system running, on a shared or recorded screen, including one case where it fails or reports uncertainty.

We do not grade video production. We grade the clarity of your reasoning. A video without you on camera, or one that only shows screens without explaining the why, does not meet this deliverable.

9. Submission

Before the deadline, send to spuentes.dev@sunnybotics.com: the repository link (with access granted if it is private), the video link, and the technical report, or open a Pull Request on the base repository if one was shared with you.

Thank you for your interest in Sunnybotics. This challenge reflects real problems our robots face in the field. Treat it as a sample of the work you would do with us.